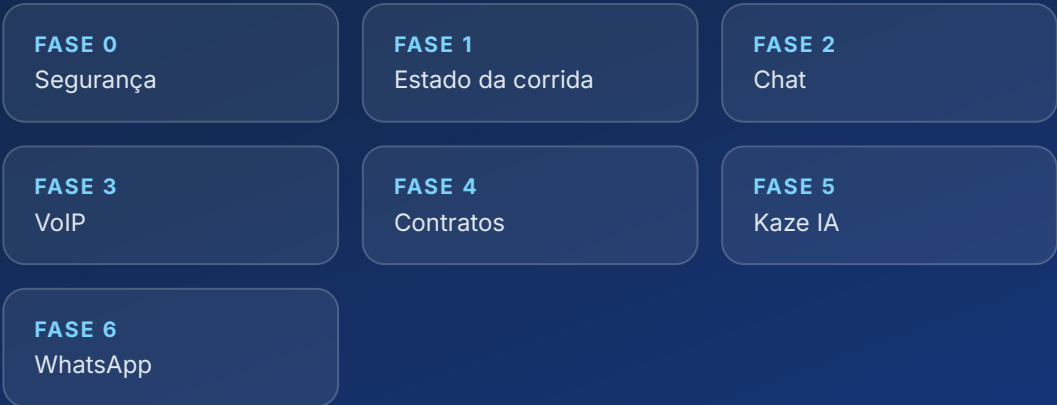


Plano Técnico de Correções

Versão revista v3.3 → v3.4

Plataforma	React 18 / TypeScript · Supabase · Capacitor
Contexto	Mobilidade urbana em Luanda (redes 3G/4G instáveis)
Data	24 de Setembro de 2026
Objectivo	Plano executável, seguro, testável e com rollback



0. O QUE MUDOU EM RELAÇÃO À VERSÃO ANTERIOR (LER PRIMEIRO)

A versão anterior tinha bom diagnóstico visual, mas **5 falhas graves de plano**:

#	Falha do plano anterior	Consequência	O que este plano faz
A	Vários fixes são sintomáticos (ex: forçar <code>accepted</code> se houver <code>driver_id</code>)	O bug real continua escondido e volta noutra forma	Corrige na origem (BD + máquina de estados) e mantém o patch só como rede de segurança
B	Zero secção de segurança	Chave Groq no bundle do frontend, canais Realtime públicos, broadcast forjável	Nova Fase 0 — Segurança (P0) antes de tudo
C	Marca tudo como " OK CORRIGIDO" sem testes nem critérios de aceitação	Ninguém sabe se está mesmo corrigido	Cada problema tem critérios de aceitação + matriz de testes
D	Sem migrações SQL, sem ordem de deploy, sem rollback	Apps Capacitor antigas em produção vão partir	Secção de deploy com compatibilidade retroactiva e feature flags
E	Links <code>file:///c:/Users/...</code> e números de linha	Inúteis para qualquer outra pessoa/IA e ficam obsoletos no próximo commit	Referências por caminho relativo + nome de função

Algumas afirmações técnicas da versão anterior estão **incorrectas** e foram corrigidas:

- **NÃO** "O WAL envia o evento antes da RPC completar o commit." → O Supabase Realtime só emite `postgres_changes` **depois do commit**. Se chega um `UPDATE` com `status='searching'` e `driver_id` preenchido, **existe outro escritor** a gravar `searching` (timer de re-despacho, cliente do passageiro, trigger, job de expiração) ou eventos fora de ordem. Isso é o que tem de ser encontrado.
- **NÃO** "O RLS rejeita silenciosamente." → O `insert` devolve `error` (código `42501` / HTTP 403). Era silencioso porque **o código não lia o `error`**.
- **NÃO** "Privacidade WhatsApp: números mascarados **OK** IMPLEMENTADO." → A WhatsApp Business API **não mascara nada** sozinha. Só há privacidade se TODO o contacto passar pelo relay do bot, e esse relay está marcado como "NÃO EXISTE" na própria tabela.
- **!** `jsPDF` com fontes padrão **não desenha emojis** (). O PDF de contratos vai sair com lixo nesses títulos.

1. PRIORIDADES E ORDEM DE EXECUÇÃO

Fase	Item	Severidade	Esforço realista	Depende de
0	Segurança: chaves no frontend, canais privados, validação admin	● P0	0,5–1 dia	—
1	Estado da corrida (fix na origem)	● P0	1–1,5 dias	Fase 0 (canais privados)
2	Chat idempotente e fiável	● P0	1 dia	Fase 0
3	VoIP Agora estável	● P0	1 dia	Fase 0
4	Contratos: origem/destino com coordenadas	● P1	1 dia	Migração SQL
5	Kaze Admin (IA)	● P1	1,5 dias	Fase 0 (chave)
6	WhatsApp Lukéni	● P2	2–3 dias	Aprovação de templates Meta
7	Observabilidade + testes transversais	● contínuo	1 dia	—

O Gantt anterior (15–40 minutos por fix) não é realista. Um fix de concorrência sem testes em rede lenta não está feito.

2. FASE 0 — SEGURANÇA (NÃO ESTAVA NO PLANO, É A MAIS URGENTE)

2.1 Chave de IA exposta no frontend

`kazeKey.ts` expõe `FRONTEND_GROQ_KEY`. Qualquer string no bundle JS/APK é pública (basta abrir o DevTools ou descompilar o APK).

Acções:

1. **Revogar e rodar** a chave Groq hoje (assumir que já foi copiada).
2. Apagar `kazeKey.ts`, `FRONTEND_GROQ_KEY`, `FRONTEND_GEMINI_KEY` e a função `callDirectJarvisChat`.
3. Chaves só em secrets das Edge Functions: `supabase secrets set GROQ_API_KEY=...`
`GEMINI_API_KEY=...`
4. Adicionar verificação no CI: falhar o build se aparecer `gsk_`, `AIza` ou `sk-` em `src/`.

```
! grep -rEn "gsk_[A-Za-z0-9]{20,}|AIza[0-9A-Za-z_-]{30,}|sk-[A-Za-z0-9]{20,}" src/
```

2.2 Canais Realtime públicos e broadcast forjável

Hoje qualquer utilizador autenticado que saiba um `rideId` pode subscrever `ride:${rideId}` / `ride_chat:${rideId}` / `call-signal:${rideId}` e enviar eventos falsos (`RIDE_UPDATED`, `NEW_MESSAGE`, `CALL_END`).

Acções: usar **canais privados** com autorização RLS em `realtime.messages`.

```
-- Só passageiro ou motorista da corrida podem ler/escrever nos tópicos da corrida
create policy "ride participants can receive"
on realtime.messages for select to authenticated
using (
  exists (
    select 1 from public.rides r
    where (realtime.topic() in ('ride:' || r.id, 'ride_chat:' || r.id, 'call:' || r.id))
    and (r.passenger_id = auth.uid() or r.driver_id = auth.uid())
  )
);

create policy "ride participants can send"
on realtime.messages for insert to authenticated
with check (
  exists (
    select 1 from public.rides r
    where (realtime.topic() in ('ride_chat:' || r.id, 'call:' || r.id))
    and (r.passenger_id = auth.uid() or r.driver_id = auth.uid())
  )
);
-- Nota: 'ride:' fica SÓ para leitura pelo cliente; quem escreve é a BD (ver Fase 1).
```

```
supabase.channel(`ride_chat:${rideId}`, { config: { private: true } })
```

2.3 Edge Function `admin-ai-proxy` sem verificação de papel

Validar no servidor que o JWT pertence a um admin (não confiar no frontend):

```
const { data: { user } } = await supabase.auth.getUser(jwt);
const { data: profile } = await supabase.from('profiles').select('role, full_name').eq('id', user?.id).single();
if (profile?.role !== 'admin') return new Response('Forbidden', { status: 403 });
```

Critérios de aceitação Fase 0

- ☐ Nenhuma chave de IA no bundle (`grep` no `dist/` e no APK vazio).
- ☐ Um utilizador que não participa na corrida recebe erro ao subscrever os canais dela.
- ☐ Um passageiro não-admin recebe 403 no `admin-ai-proxy`.

3. FASE 1 — PASSAGEIRO PRESO EM "À PROCURA"

3.1 Diagnóstico corrigido

Três problemas reais, nenhum deles é "WAL antes do commit":

1. **Outro escritor sobrescreve o estado.** Um payload com `status='searching'` + `driver_id` preenchido é um **estado impossível** que foi gravado por alguém. Suspeitos a investigar por ordem: timer de expansão de raio/re-despacho no cliente do passageiro, job/cron de expiração, trigger em `rides`, código de cancelamento do motorista que repõe `searching` sem limpar `driver_id`.
2. **Eventos fora de ordem / refetch obsoleto.** O `useRide` aplica qualquer evento que chega, mesmo que seja mais antigo que o estado actual (ex: resposta de um `select` lento a chegar depois do evento Realtime).
3. **`broadcastRideUpdated` está partido:** cria um canal novo a cada chamada (fuga de canais), ninguém do lado do passageiro escuta esse broadcast, e **depende do telemóvel do motorista** ter rede no momento certo.

3.2 Passo 1 — Descobrir o escritor (antes de corrigir)

```
-- Tabela de auditoria temporária (remover depois de 1 semana)
create table if not exists public.rides_status_audit (
  id bigserial primary key,
  ride_id uuid, old_status text, new_status text,
  old_driver uuid, new_driver uuid,
  actor uuid default auth.uid(),
  app_name text default current_setting('application_name', true),
  at timestampz default now()
);

create or replace function public.audit_rides_status() returns trigger
language plpgsql security definer as $$
begin
  if new.status is distinct from old.status or new.driver_id is distinct from old.driver_id then
    insert into public.rides_status_audit(ride_id, old_status, new_status, old_driver, new_driver)
    values (new.id, old.status, new.status, old.driver_id, new.driver_id);
  end if;
  return new;
end $$;

create trigger trg_audit_rides_status after update on public.rides
for each row execute function public.audit_rides_status();
```

Depois de reproduzir: `select * from rides_status_audit where new_status='searching' and old_driver is not null;` → mostra **quem** regrediu o estado.

3.3 Passo 2 — Tornar o estado impossível... impossível

```
-- 1) Verificar dados existentes antes de aplicar
select id, status, driver_id from public.rides where status = 'searching' and driver_id is not null;
-- 2) Corrigir esses registos manualmente, depois:
alter table public.rides
  add constraint rides_searching_has_no_driver
```

```

    check (not (status = 'searching' and driver_id is not null));

-- 3) Versão monotónica para ordenar eventos
alter table public.rides add column if not exists version bigint not null default 0;
alter table public.rides add column if not exists updated_at timestampz not null default now();

create or replace function public.bump_ride_version() returns trigger language plpgsql as $$
begin new.version := old.version + 1; new.updated_at := now(); return new; end $$;
create trigger trg_bump_ride_version before update on public.rides
for each row execute function public.bump_ride_version();

```

O escritor errado passa a **falhar com erro visível** em vez de corromper o estado.

3.4 Passo 3 — RPC atómica verdadeira

```

create or replace function public.accept_ride_atomic(p_ride_id uuid)
returns public.rides language plpgsql security definer set search_path = public as $$
declare r public.rides;
begin
    update public.rides
        set status = 'accepted', driver_id = auth.uid(), accepted_at = now()
        where id = p_ride_id and status = 'searching' and driver_id is null
    returning * into r;

    if r.id is null then
        raise exception 'RIDE_ALREADY_TAKEN' using errcode = 'P0001';
    end if;
    return r;
end $$;

```

3.5 Passo 4 — A BD emite o broadcast (não o telemóvel do motorista)

Apagar `rideService.broadcastRideUpdated`. Usar Broadcast a partir da base de dados:

```

create or replace function public.broadcast_ride_change() returns trigger
language plpgsql security definer as $$
begin
    perform realtime.send(
        jsonb_build_object('id', new.id, 'status', new.status, 'driver_id', new.driver_id,
            'driver_confirmed', new.driver_confirmed, 'version', new.version),
        'RIDE_UPDATED',
        'ride:' || new.id,
        true -- canal privado
    );
    return new;
end $$;

create trigger trg_broadcast_ride_change after update on public.rides
for each row execute function public.broadcast_ride_change();

```

3.6 Passo 5 — Cliente com máquina de estados monotónica

```

// src/hooks/useRide.ts
const lastVersionRef = useRef<number>(-1);

function applyRideUpdate(incoming: DbRide) {
    if (incoming.version <= lastVersionRef.current) return; // evento velho ou duplicado: ignora

```

```

lastVersionRef.current = incoming.version;

// Rede de segurança (mantida, mas com log para detectar regressões)
let status = (incoming.status ?? '').toLowerCase() as RideStatus;
if (incoming.driver_id && status === RideStatus.SEARCHING) {
  logger.error('ride_state_impossible', { rideId: incoming.id, version: incoming.version });
  status = RideStatus.ACCEPTED;
}

setRide(prev => ({
  ...prev,
  status,
  driverId: incoming.driver_id,
  driverConfirmed: resolveDriverConfirmed(incoming),
}));
}

// Explícito por tipo de corrida em vez de "!== false" para tudo:
function resolveDriverConfirmed(r: DbRide) {
  if (r.ride_type === 'auction') return r.driver_confirmed === true; // leilão: null = pendente
  return r.driver_confirmed !== false; // instantânea: null =
  confirmado
}

```

Subscrever **uma** fonte primária (`ride:${id}` privado) + `postgres_changes` como backup, ambos a passar por `applyRideUpdate` . Ao voltar do background (Capacitor `App.addListener('appStateChange')`) ou ao reconectar, fazer um `select` e passar também por `applyRideUpdate` .

Critérios de aceitação Fase 1

- ☐ Passageiro vê "Motorista a caminho" em < 2 s após aceitar (Wi-Fi) e < 6 s (3G simulado).
- ☐ 0 linhas com `status='searching' and driver_id is not null` durante 7 dias.
- ☐ Log `ride_state_impossible` = 0 em produção durante 7 dias (depois remover a rede de segurança).
- ☐ Dois motoristas a aceitar ao mesmo tempo: só um ganha, o outro recebe `RIDE_ALREADY_TAKEN` com mensagem clara.

4. FASE 2 — CHAT FIÁVEL E IDEMPOTENTE

4.1 Problemas que o fix anterior deixou

Problema no fix anterior	Porque é mau
<code>supabase.auth.getUser()</code> em cada mensagem	Faz chamada de rede ao servidor Auth: +300 ms a vários segundos em 3G
<code>authData?.user?.id myId</code>	Se falhar, volta a usar <code>myId</code> → reintroduz o bug do RLS
Broadcast antes de gravar	Receptor vê mensagem que pode nunca ser gravada ("mensagem fantasma")
Não lê <code>error</code> do insert	Utilizador continua sem saber que falhou
Dedupe por texto + 3 s	Duas mensagens "ok" seguidas → a segunda desaparece
Polling a cada 3 s sempre	Triplifica tráfego e bateria mesmo com Realtime saudável

4.2 Migração

```
alter table public.ride_messages add column if not exists client_id uuid;  
create unique index if not exists ride_messages_client_id_uq on public.ride_messages(client_id);
```

4.3 Envio

```
// src/components/RideChat.tsx  
const { session } = useAuth();           // id em cache (getSession), sem rede  
const senderId = session?.user.id;       // NUNCA usar myId como fallback  
  
async function send(raw: string) {  
  const text = raw.trim();  
  if (!text || !rideId || !senderId) return;  
  
  const clientId = crypto.randomUUID();  
  const msg: Msg = { id: clientId, client_id: clientId, sender_id: senderId, text,  
    created_at: new Date().toISOString(), state: 'sending' };  
  setMsgs(p => [...p, msg]);  
  setText('');  
  
  const { data, error } = await supabase  
    .from('ride_messages')  
    .insert({ ride_id: rideId, sender_id: senderId, text, client_id: clientId })  
    .select().single();  
  
  if (error) {  
    setMsgs(p => p.map(m => m.client_id === clientId ? { ...m, state: 'failed' } : m));  
    logger.warn('chat_send_failed', { code: error.code, rideId });  
    return; // UI mostra "Falhou · Tocar para reenviar"  
  }  
  
  setMsgs(p => p.map(m => m.client_id === clientId ? { ...data, state: 'sent' } : m));  
  // Broadcast só DEPOIS de gravado: rapidez sem fantasmas
```

```
channelRef.current?.send({ type: 'broadcast', event: 'NEW_MESSAGE', payload: data });
}
```

Reenvio usa o **mesmo** `client_id` → o índice único impede duplicados.

4.4 Recepção

```
function handleIncoming(m: Msg) {
  setMsgs(prev => {
    const i = prev.findIndex(x => (m.client_id && x.client_id === m.client_id) || x.id === m.id);
    if (i === -1) return [...prev, m].sort(byCreatedAt);
    const next = [...prev]; next[i] = { ...next[i], ...m, state: 'sent' }; return next;
  });
}
```

Polling só em modo degradado: activar apenas quando o canal não está `SUBSCRIBED` ou ao voltar do background, e buscar só o incremento:

```
.from('ride_messages').select('*').eq('ride_id', rideId).gt('created_at', lastSeenAt).order('created_at')
```

Critérios de aceitação Fase 2

- ☐ 100 mensagens alternadas entre 2 dispositivos: 0 perdidas, 0 duplicadas, ordem correcta.
- ☐ Modo avião a meio do envio → mensagem fica "Falhou", reenvio funciona sem duplicar.
- ☐ Mensagens iguais seguidas ("ok", "ok") aparecem ambas.

5. FASE 3 — VOIP AGORA

5.1 O fix do `callStateRef` está certo mas incompleto

`endCall` e `showToast` continuam nas dependências. Se não forem estáveis (`useCallback` com deps que mudam, ou funções vindas de contexto recriadas a cada render), **o canal continua a ser destruído** durante a chamada. Mesmo bug, outra porta.

```
const endCallRef = useLatest(endCall);
const toastRef = useLatest(showToast);

useEffect(() => {
  if (!corridaId || !userId) return;
  const ch = supabase.channel(`call:${corridaId}`, { config: { private: true } })
    .on('broadcast', { event: 'CALL_INIT' }, ({ payload }) => onInit(payload))
    .on('broadcast', { event: 'CALL_ACCEPT' }, ({ payload }) => onAccept(payload))
    .on('broadcast', { event: 'CALL_END' }, ({ payload }) => {
      if (payload.callId !== currentCallIdRef.current) return; // ignora END de chamada antiga
      endCallRef.current('remote');
    })
    .subscribe();
  signalChannelRef.current = ch;
  return () => { supabase.removeChannel(ch); signalChannelRef.current = null; };
}, [corridaId, userId]); // só isto

function useLatest<T>(v: T) { const r = useRef(v); r.current = v; return r; }
```

5.2 O que faltava por completo

1. **callId único por chamada** em todos os eventos: um `CALL_END` atrasado da chamada anterior já não mata a nova.
2. **Token Agora gerado por Edge Function** (com expiração ~1 h) e **UIDs numéricos distintos** por participante (dois `uid=0` /iguais = um expulsa o outro, sintoma idêntico a "cai ao atender").
3. **Timeouts explícitos**: tocar 30 s → "Sem resposta"; ligar ao canal Agora 15 s → erro claro. Nada de `endCall` silencioso.
4. **Permissão de microfone antes de join** (Android/Capacitor). Negada → mensagem, não chamada que cai.
5. Ouvir `connection-state-change` e `user-left` do Agora: reconectar em `RECONNECTING`, só terminar em `DISCONNECTED` com motivo.
6. Receptor com app em background não recebe broadcast → gravar `calls` na BD + push notification (FCM) para tocar.

Critérios de aceitação Fase 3

- ☐ 20 chamadas seguidas motorista↔passageiro: 0 quedas ao atender.
- ☐ Chamada de 5 min em 3G simulado sem cair; perda de rede de 5 s recupera.
- ☐ Recusar, não atender e desligar dos dois lados mostram o estado correcto nos dois ecrãs.

6. FASE 4 — CONTRATOS COM RECOLHA E DESTINO

6.1 O que faltava no plano anterior

- **Não há migração SQL.** Os campos novos existem no TypeScript mas não se vê onde são gravados.
- `address` e `dest_address` a guardar o mesmo valor = duas fontes de verdade.
- Texto livre em Luanda ("perto da cantina do Mário") não serve para despachar corrida: **faltam coordenadas**.
- 'Ponto de recolha do contrato' como `originAddr` continua a ser um texto inútil para agendar.
- Emojis no jsPDF saem como lixo.

6.2 Migração (compatível com apps antigas)

```
alter table public.contracts
  add column if not exists origin_address text,
  add column if not exists origin_lat double precision,
  add column if not exists origin_lng double precision,
  add column if not exists dest_address text,
  add column if not exists dest_lat double precision,
  add column if not exists dest_lng double precision;

-- Backfill: o antigo 'address' era o destino
update public.contracts set dest_address = address where dest_address is null;
-- 'address' fica deprecated; remover só quando nenhuma versão antiga da app estiver activa.
```

6.3 Formulário

- Dois campos com **selector no mapa / autocomplete** que grava texto + lat/lng.
- Validação: recolha e destino **obrigatórios**; recolha ≠ destino; ambos dentro da área de serviço.
- Botão "Usar a minha localização actual" para preencher a recolha.
- Deixar de escrever em `address` no formulário novo (só `dest_address`).

6.4 Agendamento

```
if (contract.origin_lat == null || contract.dest_lat == null) {
  throw new ContractIncompleteError('Contrato sem coordenadas de recolha/destino');
}
const origin = { address: contract.origin_address, lat: contract.origin_lat, lng: contract.origin_lng };
const dest = { address: contract.dest_address ?? contract.address, lat: contract.dest_lat, lng: contract.dest_lng };
```

Contratos antigos sem coordenadas: banner "Complete o ponto de recolha" em vez de falhar em silêncio.

6.5 PDF

- Remover emojis dos títulos ("PONTO DE RECOLHA", "PONTO DE DESTINO") ou embutir fonte que os suporte.

- Garantir acentos (Ã, Ç): testar com "Condomínio", "Maianga", "Cacuaco". Se falhar, embutir fonte TTF (ex: Roboto) com `addFileToVFS`.
- `splitTextToSize` para moradas longas não saírem da caixa.
- Opcional: link Google Maps com as coordenadas.

Critérios de aceitação Fase 4

- ☐ Contrato novo grava origem e destino com coordenadas.
- ☐ Contratos antigos continuam a abrir e mostram destino (backfill).
- ☐ PDF sem caracteres corrompidos, moradas longas bem quebradas.

7. FASE 5 — KAZE (IA DO PAINEL ADMIN)

7.1 Arquitectura alvo

```
KazePanel (voz/texto)
├─ Edge Function admin-ai-proxy (verifica admin, lê métricas da BD, monta prompt)
│   ├── Motor primário (modelo em env var)
│   └── Motor secundário (outro fornecedor)
└─ resposta + campo "route" + "degraded: bool"
```

Uma Edge Function, com fallback **no servidor**. O frontend deixa de ter 3 cadeias de fallback.

7.2 Correções

1. **Apagar** `callDirectJarvisChat` e a chave do frontend (Fase 0). Não é só "função morta": é uma fuga de segurança.
2. **Nomes de modelo em variáveis de ambiente** (`KAZE_PRIMARY_MODEL` , `KAZE_FALLBACK_MODEL`), validados no arranque contra o endpoint de listagem de modelos do fornecedor. Modelo inválido = erro no log, não 404 engolido.
3. **Identidade e contexto montados NO SERVIDOR**, nunca confiados ao cliente:

```
// admin-ai-proxy/index.ts
const [rides, drivers] = await Promise.all([
  db.from('rides').select('id', { count: 'exact', head: true }).in('status', ['accepted','arriving','in_progress']),
  db.from('drivers').select('id', { count: 'exact', head: true }).eq('online', true),
]);
const systemPrompt = `${SENTINEL_BASE_PROMPT}
Administrador: ${profile.full_name}. Local: Luanda, Angola. Hora local: ${nowIn('Africa/Luanda')}.
Estado actual: ${rides.count} corridas activas, ${drivers.count} motoristas online.`;
```

O cliente manda só a mensagem e o histórico. Métricas vindas do cliente podem estar obsoletas ou ser falsificadas.

4. **Acabar com a resposta fixa "Sistemas operacionais online, Comandante"** quando tudo falha. Isso esconde a avaria. Em vez disso: UI mostra "Kaze indisponível (motivo)" e botão tentar de novo; log com `route` , latência e erro.
5. **Circuit breaker** real (`consecutiveFailures` precisa de estar declarado e ser reposto a 0 no sucesso): após 3 falhas, esperar 30 s antes de voltar a chamar o motor primário.
6. **Histórico limitado** (ex: últimas 20 trocas) para não estourar tokens nem latência.

7.3 Voz (VAD) — não afinar constantes às cegas

Baixar `GATE_MIN` para 0.06 é tentativa-erro. Fazer por esta ordem:

1. `getUserMedia({ audio: { noiseSuppression: true, echoCancellation: true, autoGainControl: true } })`.
2. Mostrar **medidor de nível** em modo debug (noise floor vs voz) para afinar com dados reais.

3. **Hangover de silêncio** 900–1200 ms antes de cortar; **duração mínima** 800 ms antes de enviar para transcrição (Whisper falha com áudio curto).
4. Se continuar mau em ambiente ruidoso: trocar o VAD caseiro por um VAD neural no browser (ex: Silero VAD via [@ricky0123/vad-web](#)).
5. Transcrição também via Edge Function (sem chave no cliente).

Critérios de aceitação Fase 5

- ☐ "Quantas corridas activas temos?" devolve o número real da BD (conferir com query manual).
- ☐ "Quem és tu e onde estás?" responde Kaze Sentinel, Luanda, com nome do admin correcto.
- ☐ Com o motor primário desligado, responde via secundário e a UI indica modo degradado.
- ☐ Voz reconhecida em 9/10 frases num ambiente com ruído de rua gravado.

8. FASE 6 — WHATSAPP LUKÉNI

8.1 Correções de premissa

- Mensagens iniciadas pela empresa fora da janela de 24 h **exigem templates aprovados pela Meta**. Criar e submeter já (demora dias): `ride_accepted_passenger`, `ride_assigned_driver`.
- É preciso **opt-in** explícito do utilizador para WhatsApp (guardar `whatsapp_opt_in_at` no perfil).
- Privacidade só existe com relay completo. Até lá, **não enviar números pessoais** em nenhuma mensagem.

8.2 Disparo no servidor, não no cliente

Hoje `notifyPassengerRideAccepted` é um `fetch` do telemóvel do motorista, sem tratar erro. Se o motorista perde rede, o passageiro não é notificado.

- Disparar a partir da BD (trigger em `rides` quando `status` passa a `accepted` → fila `notifications_outbox` → Edge Function processa com retry).
- Tabela `notifications_outbox(id, ride_id, kind, status, attempts, last_error)` para idempotência e auditoria.

8.3 Relay (fase posterior)

- Mapear conversa WhatsApp ↔ `ride_id` activo; encerrar relay quando a corrida termina.
- Verificar assinatura `X-Hub-Signature-256` em todos os webhooks.
- Rate limiting e filtro de conteúdo (números de telefone, links).

Critérios de aceitação Fase 6

- ☐ Passageiro com opt-in recebe template com nome do motorista, carro e matrícula em < 10 s.
- ☐ Motorista recebe recolha, destino e primeiro nome do passageiro.
- ☐ Falha da API WhatsApp → retry automático, visível na outbox, sem afectar a corrida.

9. OBSERVABILIDADE (TRANSVERSAL)

Sem isto, "corrigido" é opinião.

- **Sentry** (ou equivalente) no app e nas Edge Functions, com `rideId` como tag.
- Eventos-chave: `ride_accept_latency_ms`, `ride_state_impossible`, `chat_send_failed`, `call_drop_reason`, `kaze_route`, `kaze_degraded`.
- Estado do canal Realtime logado em cada mudança (`SUBSCRIBED`, `CHANNEL_ERROR`, `TIMED_OUT`, `CLOSED`).
- Painel simples no admin com estas métricas das últimas 24 h.

10. MATRIZ DE TESTES

Cenário	Rede	Dispositivos	Fases
Aceitar corrida	Wi-Fi / 3G simulado / perda de pacotes 10%	2 Android reais	1
Corrida disputada por 2 motoristas	Wi-Fi	3 dispositivos	1
App do passageiro em background durante aceitação	4G	Android	1, 3
100 mensagens de chat alternadas	3G	2	2
Modo avião a meio do envio	—	1	2
20 chamadas seguidas	4G	2	3
Chamada com app fechada no receptor	4G	2	3
Contrato novo + contrato antigo + PDF	—	Web + Android	4
Kaze com motor primário desligado	—	Web	5

Ferramentas: Chrome DevTools throttling para web; Android "Network speed/latency" no emulador; testes de integração com Supabase local ([supabase start](#)) para RPC, RLS e triggers.

11. ORDEM DE DEPLOY E ROLLBACK

Apps Capacitor antigas continuam instaladas durante semanas. Tudo tem de ser **retrocompatível**.

1. **Rodar chaves** e publicar Edge Functions com secrets (Fase 0).
2. **Migrações aditivas** (colunas novas, `client_id`, `version`, auditoria, triggers). Nada é apagado.
3. Correr query de dados inválidos → corrigir → **só depois** aplicar o `CHECK` de estado.
4. Publicar Edge Functions novas (`admin-ai-proxy`, token Agora, outbox WhatsApp).
5. Publicar web; depois build Capacitor com **feature flags** (`ff_ride_broadcast_db`, `ff_chat_v2`, `ff_call_v2`) activáveis remotamente.
6. Activar flags para 10% → observar 48 h → 100%.
7. Apenas quando a versão mínima suportada for a nova: remover `address`, `broadcastRideUpdated`, canais públicos antigos, tabela de auditoria.

Rollback: cada flag desliga o comportamento novo sem novo build. Migrações são aditivas; o `CHECK` pode ser removido com um `alter table ... drop constraint` se bloquear um fluxo legítimo não previsto.

12. DEFINIÇÃO DE "FEITO" (VALE PARA CADA FASE)

- ☐ Causa raiz identificada e escrita (não só o sintoma).
- ☐ Código revisto por outra pessoa.
- ☐ Critérios de aceitação da fase todos verdes, em dispositivo real.
- ☐ Métricas/logs a confirmar em produção durante 7 dias.
- ☐ Documentação actualizada com caminhos relativos (`src/hooks/useRide.ts` > `applyRideUpdate`), sem números de linha.

13. ESTADO REAL (SUBSTITUI O "OK CORRIGIDO")

#	Problema	Estado honesto
0	Segurança (chaves, canais, admin)	NÃO Não iniciado — fazer primeiro
1	"À procura" após aceitar	● Patch sintomático aplicado; causa raiz por encontrar
2	Chat	● Melhorado; faltam idempotência, tratamento de erro, polling inteligente
3	VoIP	● Metade corrigida; <code>endCall</code> / <code>showToast</code> e <code>callId</code> em falta
4	Contratos	● UI feita; falta migração, coordenadas e PDF sem emojis
5	Kaze	EM CURSO Em curso; redesenhar para contexto no servidor
6	WhatsApp	Planeado; submeter templates Meta já

Revisão do plano v3.3. Baseado no conteúdo do documento original; validar nomes de tabelas/colunas (`rides` , `ride_messages` , `contracts` , `profiles` , `drivers`) contra o schema real antes de correr as migrações.